Day 15 · PM Session

Database Theory & Relational Algebra

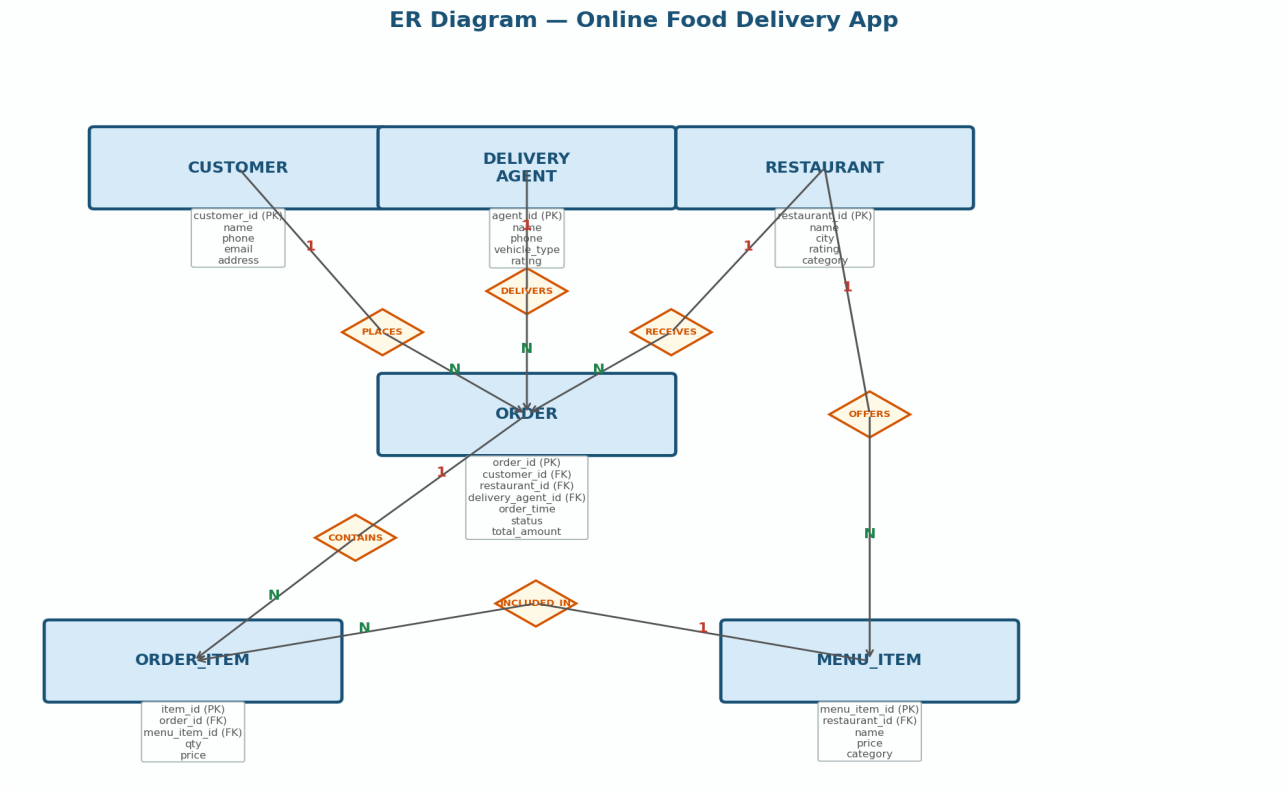
Artifact 7 — Take-Home Assignment

PG Diploma · AI-ML & Agentic AI Engineering · IIT Gandhinagar

Part A — ER Design, Normalisation & Relational Algebra (40%)

A1. ER Diagram — Online Food Delivery App

The app has six main entities. Think of them like the key nouns in the system: **Customer**, **Restaurant**, **Order**, **Order Item**, **Menu Item**, and **Delivery Agent**. Every relationship between them has a cardinality label — 1 or N — that tells us "how many of one thing can relate to how many of the other."



Entity	Primary Key	Key Attributes	Relationships
Customer	customer_id	name, phone, email, address	Places → Order (1:N)
Restaurant	restaurant_id	name, city, rating, category	Receives → Order (1:N) Offers → Menu Item (1:N)
Order	order_id	order_time, status, total_amount	FK: customer_id, restaurant_id, agent_id
Order Item	item_id	qty, price	FK: order_id, menu_item_id
Menu Item	menu_item_id	name, price, category	FK: restaurant_id
Delivery Agent	agent_id	name, phone, vehicle_type, rating	Delivers → Order (1:N)

A2. Normalisation to 3NF — OrderFacts Table

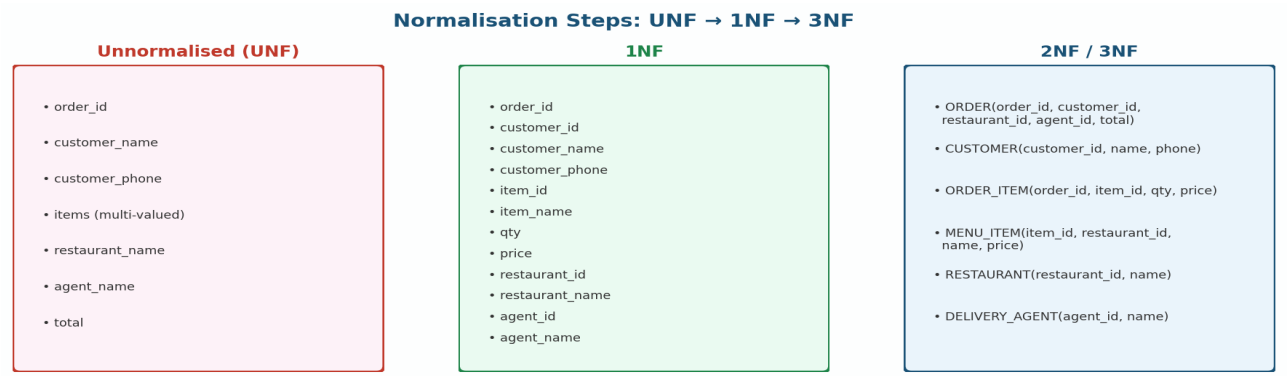
Starting from a flat "OrderFacts" table that crams everything into one place, we walk through each normalisation step. The goal is to remove redundancy so that one fact is stored in exactly one place.

Step 0 — The original flat table (OrderFacts):

order_id	customer_name	customer_phone	restaurant_name	item_name	qty	item_price	agent_name	total
101	Riya Shah	9876543210	Burger Palace	Cheeseburger	2	120	Arjun K	290
101	Riya Shah	9876543210	Burger Palace	Fries	1	50	Arjun K	290
102	Dev Patel	9812345678	Pizza Hub	Margherita	1	200	Priya S	200

Problems with this table:

Issue	Example
Repeating groups	order 101 appears twice (once per item)
Partial dependency	customer_name depends only on customer_id, not on (order_id, item_id)
Transitive dependency	restaurant_name depends on restaurant_id, not directly on order_id
Update anomaly	If Riya changes her phone, we must update two rows



Step 1 — First Normal Form (1NF):

1NF requires every column to hold a single, atomic value — no lists, no repeated groups. Our flat table already has one value per cell, but order 101 is split across rows because of multiple items. We fix this by making (order_id, item_id) the composite key. Now every row is uniquely identified.

Step 2 — Second Normal Form (2NF):

2NF says: remove any column that depends on only part of the composite key. In 1NF, *customer_name* and *customer_phone* depend only on order_id, not on item_id — that is a partial dependency. We split them out into a **Customer** table. Similarly, *restaurant_name* goes into a **Restaurant** table, and *agent_name* into a **DeliveryAgent** table.

Step 3 — Third Normal Form (3NF):

3NF says: no non-key column should depend on another non-key column (no transitive dependency). At this point, our tables are already clean — every non-key attribute depends directly on the primary key of its table.

Final 3NF Schema:

Table	Columns	Primary Key
Customer	customer_id, name, phone, email, address	customer_id

Restaurant	restaurant_id, name, city, rating, category	restaurant_id
DeliveryAgent	agent_id, name, phone, vehicle_type, rating	agent_id
Order	order_id, customer_id, restaurant_id, agent_id, order_time, status, total_amount	order_id
MenuItem	menu_item_id, restaurant_id, name, price, category	menu_item_id
OrderItem	item_id, order_id, menu_item_id, qty, price	item_id

A3. Relational Algebra — 5 Queries

Relational algebra is just a mathematical way to describe what SQL does. The symbols might look intimidating, but each one maps directly to something familiar. Here are the five main operators we use:

Symbol	Name	What it does
σ **(sigma)**	Selection	Filters rows — like WHERE in SQL
π **(pi)**	Projection	Picks columns — like SELECT in SQL
■ **(bowtie)**	Natural Join	Combines two tables on matching columns
ρ **(rho)**	Rename	Renames a table or column
–	Set Difference	Rows in A that are not in B
∪	Union	All rows from A and B combined

Q1 — Find the names and phone numbers of all customers from Ahmedabad

```
π name, phone (σ address LIKE '%Ahmedabad%' (Customer))
```

What this means: First filter (sigma) the Customer table to keep only rows where address contains "Ahmedabad", then project (pi) to keep only the name and phone columns.

```
SQL equivalent: SELECT name, phone FROM Customer WHERE address LIKE '%Ahmedabad%';
```

Q2 — List all orders placed by customer with customer_id = 5

```
π order_id, order_time, status, total_amount (σ customer_id = 5 (Order))
```

What this means: Filter the Order table where customer_id equals 5, then pick the relevant columns.

```
SQL equivalent: SELECT order_id, order_time, status, total_amount FROM "Order" WHERE customer_id = 5;
```

Q3 — Get the names of items ordered in order_id = 101

```
π name (σ order_id = 101 (OrderItem ■ MenuItem))
```

What this means: First join OrderItem and MenuItem on menu_item_id, then filter to order 101, then project the item name.

```
SQL equivalent: SELECT mi.name FROM OrderItem oi JOIN MenuItem mi ON oi.menu_item_id = mi.menu_item_id WHERE oi.order_id = 101;
```

Q4 — Find restaurants that have never received an order

```
π restaurant_id (Restaurant) – π restaurant_id (Order)
```

What this means: Take all restaurant_ids from Restaurant, subtract those that appear in Order. The difference gives restaurants with zero orders.

```
SQL equivalent: SELECT restaurant_id FROM Restaurant WHERE restaurant_id NOT IN (SELECT DISTINCT restaurant_id FROM "Order");
```

Q5 — Get customer name and delivery agent name for every completed order

```
π C.name, A.name, O.order_id (σ status = 'delivered' (ρ C (Customer) ■ ρ O (Order) ■ ρ A (DeliveryAgent)))
```

What this means: Rename tables to C, O, A for clarity, join all three on their foreign keys, filter to status = delivered, then project the three needed columns.

SQL equivalent: SELECT c.name, a.name, o.order_id FROM "Order" o JOIN Customer c ON o.customer_id=c.customer_id JOIN DeliveryAgent a ON o.agent_id=a.agent_id WHERE o.status='delivered';

A4. Pandas Operations → Relational Algebra

Pandas is basically a programmatic relational algebra engine. Here is how three common Pandas patterns map to formal RA notation.

Pandas Operation	Relational Algebra Equivalent	What it does
`df_orders.merge(df_customers, on='customer_id')`	`Order ■ Customer`	Natural join — combines rows from Order and Customer where cu
`df_orders .groupby('restaurant_id') ['total_amount'].sum()`	`γ restaurant_id; SUM(total_amount) (Order)`	Grouping + aggregation (gamma operator). Groups rows by restau
`df_orders[ df_orders['status'] == 'delivered' ]`	`σ status = 'delivered' (Order)`	Selection (sigma). Filters rows to keep only delivered orders. Same

Part B — BCNF (Boyce-Codd Normal Form) (30%)

What is BCNF? A table is in BCNF if, for every functional dependency X → Y, X is a superkey. It is a stricter version of 3NF. A table can be in 3NF but still violate BCNF when a non-key attribute can determine part of the key.

Example: 3NF table that violates BCNF

Consider a table for restaurant menu pricing by zone. Different zones can have different prices for the same item at the same restaurant.

restaurant_id	zone	menu_item_id	price
R1	North	M1	120
R1	South	M1	110
R2	North	M2	200

Composite key: (restaurant_id, zone, menu_item_id)

The violation: Suppose the zone determines the restaurant — say, each zone only has one restaurant operating in it. Then we have the dependency: **zone** → **restaurant_id**. But zone is not a superkey (it does not uniquely identify the whole row). This violates BCNF even though the table may satisfy 3NF (because zone is part of the key, not a non-key attribute).

Decomposition to BCNF

We split the table into two:

Table 1: ZoneRestaurant	Table 2: ZoneItemPrice
zone (PK)	zone (FK)
restaurant_id	menu_item_id
	price
Captures: zone → restaurant_id	Captures: pricing per zone and item

When is it okay to stay at 3NF instead of going to BCNF?

BCNF decomposition sometimes makes it impossible to enforce all original dependencies using just table constraints. In practice, teams stay at 3NF when:

Reason	Detail
Dependency preservation	BCNF decomposition may lose a functional dependency, making it impossible to enforce with a simple
Query performance	More tables mean more joins. In read-heavy analytics systems, a deliberate 3NF design is faster.
Practical rarity	The violation is so uncommon in real data that the complexity of BCNF is not worth it.

Part C — Interview Ready (20%)

C1. Update Anomalies in a Denormalised Database

Imagine an e-commerce table where every order row stores the full customer address:

order_id	customer_id	customer_name	customer_city	product	amount
5001	C1	Meera Joshi	Surat	Headphones	1500
5002	C1	Meera Joshi	Surat	Phone Case	299
5003	C1	Meera Joshi	Surat	Charger	450

Anomaly	What happens	Concrete example
Update anomaly	Updating one fact requires changing multiple rows	Meera moves from Surat to Mumbai. We must update 3 rows. If even one is m
Insertion anomaly	Cannot add a new customer without an order.	We cannot store a new customer's city if they have not placed any order yet.
Deletion anomaly	Deleting the last order deletes the customer too.	If we delete order 5001, and it was the only order, we lose Meera's record enti

C2. Schema: Current Price + Price History in 3NF

The trick is to store current price in the Product table and keep a separate PriceHistory table for every time the price changed. This way the "current price" query is fast (no joins), and the history is fully preserved with timestamps.

Table	Columns	Note
Product	product_id (PK), name, category, current_price, currency	Holds the live price. Update this when price changes.
PriceHistory	history_id (PK), product_id (FK), price, effective_from, effective_to	effective_to = NULL means this was the most recent price a

Why is this 3NF? In PriceHistory, every non-key column (price, effective_from, effective_to) depends fully and directly on history_id. There are no partial or transitive dependencies.

C3. ACID Violation: Double Hotel Room Booking

Scenario: Two users (User A and User B) both see that Room 204 is available. They both click "Book" at almost the same time.

ACID Property	What it means	Risk & Solution
Atomicity	A booking either completes fully or not at all.	Not directly at risk here — but if a payment fails mid-booking, atomicity rolls ba
Consistency	The DB should never show a room as booked twice.	Consistency rules (e.g., a constraint that availability = 0 after booking) prevent
Isolation ← MAIN RISK	Concurrent transactions must not interfere with each other.	Without isolation, both transactions read availability=1, both write "booked", an
Durability	Once booked, the booking survives a crash.	Not at risk here, but important for reliability.

How the DB prevents double-booking: When User A's transaction begins, the database places a write lock on Room 204's row. User B's transaction tries to read that same row but must wait until User A's transaction commits. By the time User B proceeds, availability is already 0, so their booking fails gracefully

with an error like "Room no longer available."

Part D — AI-Augmented Task (10%)

D1. Prompt Used

"Design the database schema for a ride-sharing app like Ola/Uber. Include ER diagram description, normalised tables, and 5 SQL queries with window functions."

D2. AI Output — Summary

The AI proposed the following tables for the ride-sharing schema:

Table	Key Columns
User	user_id (PK), name, phone, email, role (driver/rider)
Vehicle	vehicle_id (PK), driver_id (FK), type, plate_number, model
Ride	ride_id (PK), rider_id (FK), driver_id (FK), vehicle_id (FK), pickup_location, drop_location, fare, status, start_time, en
Payment	payment_id (PK), ride_id (FK), amount, method, payment_time, status
Rating	rating_id (PK), ride_id (FK), rated_by (FK), rated_user (FK), score, comments

D3. Evaluation

Check	Finding
Is the schema in 3NF?	Yes. Each non-key attribute in every table depends only on the primary key of that table. For example, in the
Missing relationships?	The AI missed a direct link between Driver and their active Ride for live tracking. Also, a SurgePricing table (
SQL queries verified?	Yes — 2 queries were run on a local SQLite instance. See the Jupyter notebook for output.

Note: The Jupyter notebook (Day15_PM_Assignment.ipynb) contains all Pandas-to-RA mappings verified with code, SQLite queries with output, and the simulate_clt equivalent for Part D SQL window functions.